parameter. The join syntax is database-specific. The following code returns a list of all line items for the book called *Programming Ruby*:

```
LineItem.select('li.quantity').
  where("pr.title = 'The Pragmatic Programmer'").
  joins("as li inner join products as pr on li.product_id = pr.id")
```

readonly

The `readonly` method causes `ActiveRecord::Resource` to return Active Record objects that cannot be stored back into the database.

If we use the `joins` or `select` method, objects will automatically be marked `readonly`.

group

The `group` method adds a `group by` clause to the SQL:

```
summary = LineItem.select("sku, sum(amount) as amount").
                   group("sku")
```

lock

The `lock` method takes an optional string as a parameter. If we pass it a string, it should be a SQL fragment in our database's syntax that specifies a kind of lock. With MySQL, for example, a *share mode* lock gives us the latest data in a row and guarantees that no one else can alter that row while we hold the lock. We could write code that debits an account only if there are sufficient funds using something like the following:

```
Account.transaction do
  ac = Account.where(id: id).lock("LOCK IN SHARE MODE").first
  ac.balance -= amount if ac.balance > amount
  ac.save
end
```